

Informe de Laboratorio 06

Tema: Arbol B+

Nota

Estudiante	Escuela	Asignatura
Sebastian Arley Chirinos Negrón schirinosne@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Estructura de datos y Algoritmos Semestre: I Código: 1702124

Laboratorio	Tema	Duración
06	Arbol B+	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2023 - A	26 Junio 2023	3 Julio 2023

1. Tarea

- Introducción En este informe, se presenta la implementación de B+.

2. URL de Repositorio Github

- URL del Repositorio GitHub para clonar o recuperar.
- <https://github.com/BastleyNait/EDA-LAB-B-23A.git>
- URL para el laboratorio 04 en el Repositorio GitHub.
- <https://github.com/BastleyNait/EDA-LAB-B-23A/tree/main/lab03>

3. Ejercicios designados

3.1. Clase B+

- Un B+ tree (árbol B+) es una estructura de datos de árbol equilibrado que se utiliza comúnmente en bases de datos y sistemas de almacenamiento para organizar y mantener grandes conjuntos de datos de manera eficiente. Es una extensión del árbol B, pero con algunas diferencias importantes que lo hacen especialmente adecuado para almacenar y recuperar datos en disco o en memoria secundaria.
- Las principales características de un B+ tree son:

- **Nodos internos y hojas:** Un B+ tree consta de nodos internos y hojas. Los nodos internos contienen claves y punteros a otros nodos internos o hojas. Las hojas contienen las claves y los datos almacenados.
- **Orden:** El B+ tree tiene un orden que se refiere al número máximo de claves que puede contener un nodo. Por ejemplo, un B+ tree de orden 3 puede tener hasta 3 claves en cada nodo.
- **Claves ordenadas:** Las claves en cada nodo están ordenadas de izquierda a derecha. Esto permite realizar búsquedas y operaciones de inserción y eliminación de manera eficiente utilizando búsqueda binaria.
- **Duplicados permitidos:** Un B+ tree permite tener claves duplicadas, lo que significa que puede haber varios elementos con la misma clave en el árbol.
- **Propiedad de búsqueda:** Todas las claves en un nodo interno actúan como "pivotes" que separan las claves de sus subárboles. Esto facilita la búsqueda y el enrutamiento a través del árbol.
- **Estructura de árbol equilibrado:** Un B+ tree siempre mantiene una altura equilibrada, lo que garantiza tiempos de búsqueda y operaciones de inserción y eliminación en el orden de $O(\log n)$, donde "n" es el número de claves almacenadas.
- **Fusión y división:** Cuando un nodo se llena o queda por debajo del mínimo permitido, se realiza la división o fusión de nodos para mantener el árbol equilibrado.
- **Debido a sus características,** los B+ trees son ampliamente utilizados en sistemas de bases de datos y almacenamiento porque ofrecen una alta eficiencia para realizar búsquedas, inserciones y eliminaciones, y permiten realizar rápidamente operaciones de rango y ordenamiento en grandes conjuntos de datos.

Listing 1: BPlustree.java

```
1 public class BPlusTree {
2     private static final int MIN_KEYS = 2; // Mnimo de claves en un nodo
3     private static final int MAX_KEYS = 4; // Mximo de claves en un nodo
4
5     private BPlusNode root;
6
7     // Constructor de BPlusTree
8     public BPlusTree() {
9         root = new BPlusNode();
10    }
11
12    public void printTree() {
13        if (root != null) {
14            printTreeRecursive(root, 0);
15        }
16    }
17
18    private void printTreeRecursive(BPlusNode node, int level) {
19        if (node == null) {
20            return;
21        }
22
23        for (int i = 0; i < node.keys.size(); i++) {
24            if (!node.isLeaf) {
25                printTreeRecursive(node.children.get(i), level + 1);
```

```
26     }
27     printNodeInfo(node.keys.get(i), level);
28 }
29
30 // Imprimir el ltimo hijo (si existe) de un nodo no hoja
31 if (!node.isLeaf) {
32     printTreeRecursive(node.children.get(node.keys.size()), level + 1);
33 }
34 }
35
36 private void printNodeInfo(int key, int level) {
37     StringBuilder indent = new StringBuilder();
38     for (int i = 0; i < level; i++) {
39         indent.append(" "); // Dos espacios por nivel para indentacin
40     }
41     System.out.println(indent.toString() + key);
42 }
43
44 // Mtodo para insertar una nueva clave en el rbol
45 public void insert(int key) {
46     BPlusNode leafNode = findLeafNode(key);
47     leafNode.insertKey(key);
48     leafNode.balanceTree();
49 }
50
51 // Mtodo auxiliar para encontrar el nodo hoja donde debe insertarse la clave
52 private BPlusNode findLeafNode(int key) {
53     BPlusNode currentNode = root;
54     while (!currentNode.isLeaf) {
55         int index = 0;
56         while (index < currentNode.keys.size() && key >= currentNode.keys.get(index)) {
57             index++;
58         }
59         currentNode = currentNode.children.get(index);
60     }
61     return currentNode;
62 }
63
64 // Mtodo para buscar una clave en el rbol
65 public boolean search(int key) {
66     BPlusNode currentNode = root;
67     while (!currentNode.isLeaf) {
68         int index = 0;
69         while (index < currentNode.keys.size() && key >= currentNode.keys.get(index)) {
70             index++;
71         }
72         currentNode = currentNode.children.get(index);
73     }
74
75     // Buscar la clave en el nodo hoja actual
76     int index = 0;
77     while (index < currentNode.keys.size() && key > currentNode.keys.get(index)) {
78         index++;
79     }
80
81     return index < currentNode.keys.size() && currentNode.keys.get(index) == key;
```

```
82 }  
83 }  
84 }
```

- Las principales características de un B+ tree son:
- Nodos internos y hojas: Un B+ tree consta de nodos internos y hojas. Los nodos internos contienen claves y punteros a otros nodos internos o hojas. Las hojas contienen las claves y los datos almacenados.
- Orden: El B+ tree tiene un orden que se refiere al número máximo de claves que puede contener un nodo. Por ejemplo, un B+ tree de orden 3 puede tener hasta 3 claves en cada nodo.
- Claves ordenadas: Las claves en cada nodo están ordenadas de izquierda a derecha. Esto permite realizar búsquedas y operaciones de inserción y eliminación de manera eficiente utilizando búsqueda binaria.
- Duplicados permitidos: Un B+ tree permite tener claves duplicadas, lo que significa que puede haber varios elementos con la misma clave en el árbol.
- Propiedad de búsqueda: Todas las claves en un nodo interno actúan como "pivotes" que separan las claves de sus subárboles. Esto facilita la búsqueda y el enrutamiento a través del árbol.
- Estructura de árbol equilibrado: Un B+ tree siempre mantiene una altura equilibrada, lo que garantiza tiempos de búsqueda y operaciones de inserción y eliminación en el orden de $O(\log n)$, donde n es el número de claves almacenadas.
- Fusión y división: Cuando un nodo se llena o queda por debajo del mínimo permitido, se realiza la división o fusión de nodos para mantener el árbol equilibrado.
- Debido a sus características, los B+ trees son ampliamente utilizados en sistemas de bases de datos y almacenamiento porque ofrecen una alta eficiencia para realizar búsquedas, inserciones y eliminaciones, y permiten realizar rápidamente operaciones de rango y ordenamiento en grandes conjuntos de datos.

3.2. Clase BPlusNode

- En resumen, esta clase Nodo representa un nodo en un árbol binario, utilizado en el contexto del BPlustree. Cada nodo contiene un dato, referencias a los nodos hijo izquierdo y derecho, y un campo para almacenar la altura del nodo. Proporciona métodos para acceder y modificar estos campos, así como métodos para obtener representaciones del nodo y realizar diferentes recorridos en profundidad en el árbol.

Listing 2: BPlusNode.java

```
1 import java.util.*;  
2 public class BPlusNode {  
3     private static final int MAX_KEYS = 4;  
4     private static final int MIN_KEYS = 2;  
5     // Lista de de claves que tendra un nodo  
6     // No los hago privados para no hacer un codigo tan extenso  
7     List<Integer> keys;  
8     // Lista de punteros a otros nodos  
9     List<BPlusNode> children;  
10 }
```

```
11 //Referencia al nodo padre
12 BPlusNode parent;
13
14 //false si es interno y true si es hoja
15
16 boolean isLeaf;
17
18 public BPlusNode(){
19     this.keys = new ArrayList<>();
20     this.children = new ArrayList<>();
21     parent = null;
22     isLeaf = true;
23 }
24
25
26 // Mtodo para dividir el nodo en dos
27 public BPlusNode split() {
28     // Creamos un nuevo nodo que ser el nodo derecho despues de la divisin
29     BPlusNode rightNode = new BPlusNode();
30
31     // Calculamos el ndice donde se dividir el nodo actual
32     int midIndex = keys.size() / 2;
33
34     // Movemos la mitad derecha de las claves y punteros al nuevo nodo
35     rightNode.keys.addAll(keys.subList(midIndex, keys.size()));
36     keys.subList(midIndex, keys.size()).clear();
37
38     // Movemos los punteros si el nodo es interno
39     if (!isLeaf) {
40         rightNode.children.addAll(children.subList(midIndex, children.size()));
41         children.subList(midIndex, children.size()).clear();
42     }
43
44     // Actualizamos el nodo padre para ambos nodos (si existe)
45     if (parent != null) {
46         rightNode.parent = parent;
47         int insertIndex = parent.findInsertIndex(rightNode.keys.get(0));
48         parent.children.add(insertIndex + 1, rightNode);
49     }
50
51     // Devolvemos el nuevo nodo derecho creado despues de la divisin
52     return rightNode;
53 }
54
55 // Mtodo para encontrar la posicin de insercin de una clave en el nodo
56 int findInsertIndex(int key) {
57     int insertIndex = 0;
58     while (insertIndex < keys.size() && key > keys.get(insertIndex)) {
59         insertIndex++;
60     }
61     return insertIndex;
62 }
63
64 public void mergeWith(BPlusNode rightNode) {
65     // Movemos todas las claves y punteros del nodo derecho al nodo actual
66     keys.addAll(rightNode.keys);
```

```
67     children.addAll(rightNode.children);
68
69     // Actualizamos el padre del nodo derecho para que apunte al nodo actual
70     for (BPlusNode child : rightNode.children) {
71         child.parent = this;
72     }
73
74     // Borramos la referencia del nodo derecho en el padre (ya no ser necesario)
75     if (parent != null) {
76         parent.children.remove(rightNode);
77     }
78 }
79
80 // Metodo para encontrar el ndice del nodo vecino derecho
81 public int findRightNeighborIndex() {
82     if (parent == null) {
83         return -1; // No hay nodo vecino derecho (el nodo actual es la raz)
84     }
85     return parent.children.indexOf(this) + 1;
86 }
87
88 // Metodo para encontrar el ndice del nodo vecino izquierdo
89 public int findLeftNeighborIndex() {
90     if (parent == null) {
91         return -1; // No hay nodo vecino izquierdo (el nodo actual es la raz)
92     }
93     return parent.children.indexOf(this) - 1;
94 }
95 // Metodo para obtener la clave central
96 public int getCentralKey() {
97     int midIndex = keys.size() / 2;
98     return keys.get(midIndex);
99 }
100
101 public int findKeyPosition(int key) {
102     int left = 0;
103     int right = keys.size() - 1;
104
105     while (left <= right) {
106         int mid = left + (right - left) / 2;
107         int midKey = keys.get(mid);
108
109         if (midKey == key) {
110             return mid; // Encontramos la clave en la posicin 'mid'
111         } else if (midKey < key) {
112             left = mid + 1;
113         } else {
114             right = mid - 1;
115         }
116     }
117
118     // Si no encontramos la clave, devolvemos la posicin donde debera estar
119     return left;
120 }
121
122 public List<Integer> getAllKeys() {
```

```
123     return this.keys;
124 }
125
126 // Mtodo para imprimir el contenido del nodo y sus hijos
127 public void printNode() {
128     System.out.print("Keys: ");
129     for (int key : keys) {
130         System.out.print(key + " ");
131     }
132     System.out.println();
133
134     if (!isLeaf) {
135         System.out.print("Children: ");
136         for (BPlusNode child : children) {
137             System.out.print(child.getCentralKey() + " ");
138         }
139         System.out.println();
140
141         for (BPlusNode child : children) {
142             child.printNode();
143         }
144     }
145 }
146
147 public void balanceTree() {
148     if (keys.size() <= MAX_KEYS) {
149         // El nodo est equilibrado, no es necesario hacer nada
150         return;
151     }
152
153     if (parent == null) {
154         // Si el nodo es la raz y ha alcanzado el mximo de claves permitidas,
155         // creamos un nuevo nodo como la nueva raz y dividimos el nodo actual
156         BPlusNode newRoot = new BPlusNode();
157         newRoot.children.add(this);
158         parent = newRoot;
159         newRoot.split();
160         return;
161     }
162
163     // Si el nodo est desequilibrado y no es la raz, dividimos el nodo y
164     // realizamos la insercin del nodo resultado en el padre
165     parent.split();
166 }
167 public void insertKey(int key) {
168     int index = 0;
169     while (index < keys.size() && key > keys.get(index)) {
170         index++;
171     }
172     keys.add(index, key);
173     if (keys.size() > MAX_KEYS) {
174         split();
175     }
176 }
177
178 }
```

- Este código implementa la estructura de un árbol B+ en Java utilizando listas enlazadas para los nodos y punteros.
- Un árbol B+ es una variante de un árbol B que se utiliza comúnmente en bases de datos y sistemas de almacenamiento para organizar y buscar grandes conjuntos de datos de manera eficiente.
- En esta implementación, la clase BPlusNode representa un nodo del árbol B+. Cada nodo contiene una lista de claves y una lista de punteros a otros nodos. Los nodos pueden ser nodos hoja (isLeaf = true) que almacenan datos o nodos internos (isLeaf = false) que se utilizan para organizar la estructura del árbol.

3.3. Clase Main

- La clase main utiliza el árbol b+ para generar y manipular un árbol B+.

Listing 3: Main.java

```
1 import java.util.Scanner;
2
3 public class Main {
4     static Scanner sc = new Scanner(System.in);
5     public static void main(String[] args) {
6         BPlusTree bpt = new BPlusTree();
7         insertarKeys(bpt);
8         bpt.printTree();
9         buscar(bpt);
10    }
11
12    public static void insertarKeys(BPlusTree tree){
13        int c = 0;
14        System.out.println("Cuantos elementos desea insertar en el arbol b+");
15        int n = sc.nextInt();
16        System.out.println("Intruzca los valores");
17        while(c<n){
18            int key = sc.nextInt();
19            tree.insert(key);
20            c++;
21        }
22    }
23
24    public static void buscar(BPlusTree tree) {
25        System.out.println("que valor desea buscar?: ");
26        int key = sc.nextInt();
27        boolean bool = tree.search(key);
28        if (bool) {
29            System.out.printf("\nLa key " + key + " s se encuentra en el arbol");
30        }
31        else {
32            System.out.printf("\nLa key " + key + " no se encuentra en el arbol");
33        }
34    }
35 }
```

- Este código es un programa principal en Java que utiliza la implementación de un árbol B+ (BPlusTree) para insertar claves y buscar valores en el árbol.

- El programa comienza creando una instancia de BPlusTree llamada bpt. Luego, invoca el método insertarKeys() para insertar claves en el árbol bpt a través de la entrada del usuario. Después de insertar las claves, se llama al método printTree() para imprimir el contenido del árbol.
- A continuación, el programa solicita al usuario que ingrese un valor para buscar en el árbol utilizando el método buscar(). El valor ingresado se pasa al método search() del árbol bpt, que realiza una búsqueda en el árbol y devuelve un valor booleano que indica si la clave se encuentra o no en el árbol. El resultado de la búsqueda se imprime en la consola.
- En resumen, este programa permite al usuario insertar claves en un árbol B+ y luego buscar valores específicos en el árbol.

3.4. Commits del trabajo

- Aquí están algunos de los commits mas importantes en este trabajo:

```
PS C:\Users\Sewas\Desktop\LABS\EDA-LAB-B-23A\lab06> git log
commit 7b2f262d1322ddb996cfbb1a535e5bc06437f722 (HEAD -> main)
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Mon Jul 3 15:04:13 2023 -0500

    implimentacion de eliminacion

commit 39a9f19c677e78adf48ed0cccf565764dfdfd408
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Mon Jul 3 09:07:57 2023 -0500

    implementando busqueda

commit 1d2fd2e324960fd443bd68d0e68256f6f27f70ab
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Mon Jul 3 09:05:15 2023 -0500

    creando la clase bplustree

commit a74817b71450463a7cb47727e28d185ab7b04df3
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Mon Jul 3 08:59:58 2023 -0500

    creando un nodo mejor elaborado

commit cc93b96d97b5ea5d98de566e623e78968bdb0686
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Sun Jul 2 23:41:32 2023 -0500

    arreglando errores
!...skipping...
commit 7b2f262d1322ddb996cfbb1a535e5bc06437f722 (HEAD -> main)
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Mon Jul 3 15:04:13 2023 -0500

    implimentacion de eliminacion

commit 39a9f19c677e78adf48ed0cccf565764dfdfd408
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Mon Jul 3 09:07:57 2023 -0500

    implementando busqueda

commit 1d2fd2e324960fd443bd68d0e68256f6f27f70ab
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Mon Jul 3 09:05:15 2023 -0500

    creando la clase bplustree

commit a74817b71450463a7cb47727e28d185ab7b04df3
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Mon Jul 3 08:59:58 2023 -0500

    creando un nodo mejor elaborado

commit cc93b96d97b5ea5d98de566e623e78968bdb0686
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Sun Jul 2 23:41:32 2023 -0500

    arreglando errores

commit 7b2f262d1322ddb996cfbb1a535e5bc06437f722 (HEAD -> main)
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Mon Jul 3 15:04:13 2023 -0500

    implimentacion de eliminacion

commit 39a9f19c677e78adf48ed0cccf565764dfdfd408
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Mon Jul 3 09:07:57 2023 -0500
```

3.5. Estructura de laboratorio 06

- El contenido que se entrega en este laboratorio es el siguiente:

```
lab06/
| .gitignore
| BPlustree.java
| BPlusNode.java
| Main.java

\---latex
| EDA_lab06_schirinosne.pdf
| EDA_lab06_schirinosne.tex
|
+---img
| Commits.png
| logo_abet.png
| logo_episunsa.png
| logo_unsa.jpg
|
\---src
| BPlustree.java
| BPlusNode.java
| Main.java
```

4. Rúbricas

4.1. Entregable Informe

Tabla 1: Tipo de Informe

Informe	
Latex	El informe está en formato PDF desde Latex, con un formato limpio (buena presentación) y facil de leer.

4.2. Rúbrica para el contenido del Informe y demostración

- El alumno debe marcar o dejar en blanco en celdas de la columna **Checklist** si cumple con el ítem correspondiente.
- Si un alumno supera la fecha de entrega, su calificación será sobre la nota mínima aprobada, siempre y cuando cumpla con todos lo ítems.
- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la siguiente tabla:

Tabla 2: Niveles de desempeño

	Nivel			
Puntos	Insatisfactorio 25 %	En Proceso 50 %	Satisfactorio 75 %	Sobresaliente 100 %
2.0	0.5	1.0	1.5	2.0
4.0	1.0	2.0	3.0	4.0

Tabla 3: Rúbrica para contenido del Informe y demostración

	Contenido y demostración	Puntos	Checklist	Estudiante	Profesor
1. GitHub	Hay enlace URL activo del directorio para el laboratorio hacia su repositorio GitHub con código fuente terminado y fácil de revisar.	2	X	2	
2. Commits	Hay capturas de pantalla de los commits más importantes con sus explicaciones detalladas. (El profesor puede preguntar para refrendar calificación).	4	x	4	
3. Código fuente	Hay porciones de código fuente importantes con numeración y explicaciones detalladas de sus funciones.	2	X	2	
4. Ejecución	Se incluyen ejecuciones/pruebas del código fuente explicadas gradualmente.	2	X	2	
5. Pregunta	Se responde con completitud a la pregunta formulada en la tarea. (El profesor puede preguntar para refrendar calificación).	2	X	2	
6. Fechas	Las fechas de modificación del código fuente estan dentro de los plazos de fecha de entrega establecidos.	2	X	2	
7. Ortografía	El documento no muestra errores ortográficos.	2	X	2	
8. Madurez	El Informe muestra de manera general una evolución de la madurez del código fuente, explicaciones puntuales pero precisas y un acabado impecable. (El profesor puede preguntar para refrendar calificación).	4	x	2	
Total		20		18	

5. Referencias

- <https://www.geeksforgeeks.org/introduction-to-avl-tree/>
- <https://www.geeksforgeeks.org/insertion-in-an-avl-tree/>
- <https://www.geeksforgeeks.org/deletion-in-an-avl-tree/>
- <https://docs.oracle.com/javase/tutorial/java/generics/types.html>
- <https://algorithmtutor.com/Data-Structures/Tree/AVL-Trees/>